

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO
PAULO**

VINÍCIUS DE MORAIS ALMEIDA SANTOS

**ESTUDO DE CASO UTILIZANDO MONGODB PARA A MODELAGEM NÃO
RELACIONAL DE UM E-COMMERCE DE VESTUÁRIO**

**CAMPOS DO JORDÃO
2026**

VINÍCIUS DE MORAIS ALMEIDA SANTOS

**ESTUDO DE CASO UTILIZANDO MONGODB PARA A MODELAGEM NÃO
RELACIONAL DE UM E-COMMERCE DE VESTUÁRIO**

Relatório apresentado como requisito parcial de avaliação da disciplina de Banco de Dados II (BDD2), referente ao Projeto Final – NoSQL, do curso superior de Tecnologia em Análise e Desenvolvimento de Sistemas do Instituto Federal de Educação, Ciência e Tecnologia de São Paulo – Campus Campos do Jordão.

Orientador: Paulo Giovani de Faria
Zeferino

CAMPOS DO JORDÃO

2026

RESUMO

A migração de arquiteturas relacionais para modelos não relacionais tem se consolidado como estratégia relevante em cenários de e-commerce que demandam flexibilidade de esquema, alta disponibilidade e escalabilidade horizontal. O presente trabalho tem como objetivo remodelar, sob a abordagem NoSQL orientada a documentos, o banco de dados de uma loja virtual de vestuário previamente concebido em modelo relacional no Microsoft SQL Server. Utilizou-se o Sistema de Gerenciamento de Banco de Dados MongoDB, adotando-se a linguagem de consulta baseada em documentos BSON e o recurso de Aggregation Pipeline para a extração de métricas analíticas. A metodologia contemplou o estudo comparativo entre os modelos de dados NoSQL existentes (chave-valor, orientado a documentos, colunar e orientado a grafos), a justificativa da escolha do modelo orientado a documentos, a transposição das seis entidades normalizadas do projeto relacional anterior (Cliente, Categoria, Produto, Pedido, Item_Pedido e Pagamento) para três coleções desnormalizadas (clientes, produtos e pedidos), e a reutilização da mesma carga de dados originalmente injetada no ambiente relacional, compreendendo 100 clientes, 30 produtos e 1.000 pedidos. Como resultado, apresentam-se as decisões de modelagem de dados adotadas, com ênfase na estratégia de incorporação (embedding) de itens de pedido e pagamento dentro do próprio documento de pedido, e um conjunto de 30 consultas analíticas construídas por meio do framework de agregação do MongoDB, contemplando operadores como \$match, \$group, \$lookup, \$unwind, \$facet e \$setWindowFields, equivalentes funcionais das consultas SQL avançadas do projeto anterior. Conclui-se que o modelo orientado a documentos favorece a performance de leitura em operações que privilegiam a recuperação do pedido completo em uma única consulta, reduzindo a necessidade de junções, ao passo que introduz desafios relacionados à consistência transacional multidocumento e à governança de dados desnormalizados, aspectos que devem ser cuidadosamente ponderados na escolha entre paradigmas relacional e não relacional.

Palavras-Chave: Banco de Dados NoSQL; MongoDB; Modelagem Orientada a Documentos; E-commerce; Aggregation Pipeline.

ABSTRACT

The migration from relational architectures to non-relational models has become a relevant strategy in e-commerce scenarios that demand schema flexibility, high availability, and horizontal scalability. The present work aims to remodel, under a document-oriented NoSQL approach, the database of an online clothing store previously designed using a relational model in Microsoft SQL Server. The MongoDB Database Management System was used, adopting the BSON document-based query language and the Aggregation Pipeline feature for extracting analytical metrics. The methodology included a comparative study of existing NoSQL data models (key-value, document-oriented, columnar, and graph-oriented), the justification for choosing the document-oriented model, the transposition of the six normalized entities from the previous relational project (Client, Category, Product, Order, Order_Item, and Payment) into three denormalized collections (clients, products, and orders), and the reuse of the same dataset originally injected into the relational environment, comprising 100 clients, 30 products, and 1,000 orders. As a result, the data modeling decisions adopted are presented, emphasizing the embedding strategy for order items and payment within the order document itself, along with a set of 30 analytical queries built using the MongoDB aggregation framework, including operators such as \$match, \$group, \$lookup, \$unwind, \$facet, and \$setWindowFields, functional equivalents of the advanced SQL queries from the previous project. It is concluded that the document-oriented model favors read performance in operations that prioritize retrieving the complete order in a single query, reducing the need for joins, while introducing challenges related to multi-document transactional consistency and denormalized data governance, aspects that must be carefully weighed when choosing between relational and non-relational paradigms.

Keywords: NoSQL Database; MongoDB; Document-Oriented Modeling; E-commerce; Aggregation Pipeline.

LISTA DE ILUSTRAÇÕES

FIGURA 1 – Modelo de Documentos do Banco de Dados NoSQL	17
FIGURA 2 – Faturamento Total Acumulado por Categoria	23
FIGURA 3 – Ranking dos 5 Clientes com Maior Volume de Compras	24
FIGURA 4 – Produtos com Estoque Crítico	25
FIGURA 5 – Taxa de Rejeição: Percentual de Pedidos Cancelados	26
FIGURA 6 – Distribuição de Receita por Método de Pagamento	26
FIGURA 7 – Análise de Vendas Cruzadas / Cross-Selling	27
FIGURA 8 – Desempenho de Vendas por Estado (UF)	28
FIGURA 9 – Curva de Retenção: Clientes Inativos	29
FIGURA 10 – Sazonalidade do Faturamento	30
FIGURA 11 – Os 3 Produtos Menos Vendidos	31
FIGURA 12 – Campeões de Venda por Categoria	32
FIGURA 13 – Cohort de Clientes: Tempo Retido	33
FIGURA 14 – Faturamento Acumulado Anual / Running Total	34
FIGURA 15 – Pedidos Premium: Acima da Média Nacional	35
FIGURA 16 – Matriz de Status de Pedidos por Ano	36
FIGURA 17 – Capital Imobilizado em Estoque por Categoria	37
FIGURA 18 – Produtos Órfãos: Nunca Comercializados	37
FIGURA 19 – Perfil Demográfico: Domínio de E-mail	38
FIGURA 20 – Média de Dias para Recompra	39

FIGURA 21 – Market Share Estadual	40
FIGURA 22 – Ranking de Ticket Médio por Estado	41
FIGURA 23 – Crescimento Financeiro Mês a Mês	42
FIGURA 24 – Os 3 Produtos com Maior Impacto Financeiro	43
FIGURA 25 – A Maior Transação Única do Sistema	44
FIGURA 26 – Tempo Médio entre Pedido e Pagamento	44
FIGURA 27 – Matriz Demográfica: O que cada Estado Compra	45
FIGURA 28 – Frequência de Compras Repetidas por Cliente	46
FIGURA 29 – Valor da Oportunidade Perdida: Pedidos Pendentes	46
FIGURA 30 – Produtos Cadastrados vs. Disponíveis	47
FIGURA 31 – Visão Desnormalizada de Auditoria	48

LISTA DE QUADROS

QUADRO 1 - Transposição do Modelo Relacional para o Modelo de Documentos	17
--	----

LISTA DE ALGORITMOS

- ALGORITMO A** – Exemplo de Documento da Coleção clientes
- ALGORITMO B** – Exemplo de Documento da Coleção produtos
- ALGORITMO C** – Exemplo de Documento da Coleção pedidos
- ALGORITMO D** – Script de Geração Massiva de Pedidos em MongoDB
- ALGORITMO 1** – Faturamento Total Acumulado por Categoria
- ALGORITMO 2** – Ranking dos 5 Clientes com Maior Volume de Compras
- ALGORITMO 3** – Produtos com Estoque Crítico
- ALGORITMO 4** – Taxa de Rejeição: Percentual de Pedidos Cancelados
- ALGORITMO 5** – Distribuição de Receita por Método de Pagamento
- ALGORITMO 6** – Análise de Vendas Cruzadas / Cross-Selling
- ALGORITMO 7** – Desempenho de Vendas por Estado (UF)
- ALGORITMO 8** – Curva de Retenção: Clientes Inativos
- ALGORITMO 9** – Sazonalidade do Faturamento
- ALGORITMO 10** – Os 3 Produtos Menos Vendidos
- ALGORITMO 11** – Campeões de Venda por Categoria
- ALGORITMO 12** – Cohort de Clientes: Tempo Retido
- ALGORITMO 13** – Faturamento Acumulado Anual / Running Total
- ALGORITMO 14** – Pedidos Premium: Acima da Média Nacional
- ALGORITMO 15** – Matriz de Status de Pedidos por Ano
- ALGORITMO 16** – Capital Imobilizado em Estoque por Categoria
- ALGORITMO 17** – Produtos Órfãos (Nunca Comercializados)
- ALGORITMO 18** – Perfil Demográfico: Domínio de E-mail
- ALGORITMO 19** – Média de Dias para Recompra
- ALGORITMO 20** – Market Share Estadual
- ALGORITMO 21** – Ranking de Ticket Médio por Estado
- ALGORITMO 22** – Crescimento Financeiro Mês a Mês
- ALGORITMO 23** – Os 3 Produtos com Maior Impacto Financeiro
- ALGORITMO 24** – A Maior Transação Única do Sistema
- ALGORITMO 25** – Tempo Médio entre Pedido e Pagamento
- ALGORITMO 26** – Matriz Demográfica: O que cada Estado Compra
- ALGORITMO 27** – Frequência de Compras Repetidas por Cliente
- ALGORITMO 28** – Valor da Oportunidade Perdida (Pedidos Pendentes)

ALGORITMO 29 – Produtos Cadastrados vs. Disponíveis

ALGORITMO 30 – Visão Desnormalizada de Auditoria

LISTA DE SIGLAS

ACID	Atomicidade, Consistência, Isolamento e Durabilidade
BSON	Binary JSON
CAP	Consistência, Disponibilidade e Tolerância a Partições (Teorema CAP)
CRUD	Create, Read, Update, Delete
JSON	JavaScript Object Notation
NoSQL	Not Only SQL
SGBD	Sistema de Gerenciamento de Banco de Dados
TTL	Time To Live
UF	Unidade Federativa

SUMÁRIO

1 INTRODUÇÃO	12
1.1 Objetivos	12
1.2 Justificativa	13
1.3 Aspectos Metodológicos	14
1.4 Aporte Teórico	14
2 FUNDAMENTAÇÃO TEÓRICA	15
2.1 Bancos de Dados NoSQL e o Teorema CAP	15
2.2 Modelo Orientado a Documentos e o MongoDB	15
2.3 Trabalhos Relacionados	16
3 PROJETO PROPOSTO	17
3.1 Considerações Iniciais	17
3.2 Do Modelo Relacional ao Modelo de Documentos	17
3.3 Modelagem de Dados: Coleções e Documentos	18
3.3.1 Coleção clientes	18
3.3.2 Coleção produtos	19
3.3.3 Coleção pedidos	19
3.4 Estratégia de Carga de Dados (Injeção)	21
4 AVALIAÇÃO	22
4.1 Consultas Analíticas (Aggregation Pipeline)	22
5 CONCLUSÃO	48
REFERÊNCIAS	51

1 INTRODUÇÃO

Os bancos de dados não relacionais, popularmente designados pelo termo NoSQL (Not Only SQL), surgiram como resposta às limitações do modelo relacional tradicional diante de cenários de grande volume, velocidade e variedade de dados — características típicas da chamada era do Big Data. Diferentemente do modelo relacional, que impõe esquemas rígidos e privilegia a normalização como mecanismo de integridade, os bancos NoSQL priorizam a flexibilidade de esquema, a escalabilidade horizontal e, em muitos casos, a desnormalização deliberada como estratégia de otimização de leitura.

Existem, atualmente, quatro grandes famílias de bancos de dados NoSQL amplamente reconhecidas na literatura: os bancos chave-valor (key-value), como o Redis e o DynamoDB, que armazenam dados como pares simples de identificador e valor, sendo ideais para caching e sessões; os bancos orientados a documentos, como o MongoDB e o CouchDB, que armazenam registros semiestruturados em formato JSON/BSON, permitindo estruturas aninhadas e arrays; os bancos colunares (wide-column), como o Cassandra e o HBase, otimizados para escrita massiva e consultas analíticas sobre grandes volumes distribuídos; e os bancos orientados a grafos, como o Neo4j, voltados à modelagem de relacionamentos complexos entre entidades, como redes sociais e sistemas de recomendação.

Este trabalho tem por objetivo remodelar o banco de dados de um e-commerce de vestuário e acessórios — anteriormente projetado em modelo relacional normalizado no Microsoft SQL Server, como entregue no Projeto Final de Banco de Dados Relacional — sob a ótica de um SGBD não relacional orientado a documentos, adotando o MongoDB como ferramenta de implementação. Buscou-se preservar o domínio de negócio (cadastro de clientes, catálogo de produtos, pedidos, itens de pedido e pagamentos), avaliando de que forma a ausência de esquema fixo e a possibilidade de aninhamento de documentos alteram as decisões de projeto de dados.

1.1 Objetivos

Este trabalho tem por objetivo geral projetar e implementar, em ambiente MongoDB, uma base de dados não relacional orientada a documentos para o gerenciamento das operações de um e-commerce de vestuário, avaliando

comparativamente as implicações dessa escolha frente ao modelo relacional previamente desenvolvido.

Para a consecução deste objetivo, foram estabelecidos os seguintes objetivos específicos:

- Estudar os modelos de dados NoSQL existentes (chave-valor, documento, colunar e grafos), justificando a escolha do modelo orientado a documentos para o domínio de e-commerce;
- Transpor as seis entidades relacionais normalizadas (Cliente, Categoria, Produto, Pedido, Item_Pedido e Pagamento) para uma estrutura de coleções desnormalizadas no MongoDB, aplicando estratégias de embedding (incorporação) e referencing (referenciamento) conforme a natureza de cada relacionamento;
- Popular as coleções com uma carga de dados massiva e equivalente à do projeto relacional anterior (100 clientes, 30 produtos e 1.000 pedidos);
- Elaborar e validar 30 consultas analíticas complexas utilizando o Aggregation Framework do MongoDB, reproduzindo funcionalmente as métricas de Business Intelligence extraídas anteriormente via T-SQL;
- Discutir as vantagens, limitações e trade-offs do modelo não relacional adotado em comparação ao modelo relacional.

1.2 Justificativa

A escolha do domínio de e-commerce de vestuário para este estudo de caso não é arbitrária: trata-se de um cenário no qual convivem entidades com alta variabilidade de atributos — como produtos que podem apresentar tamanho, cor e outras variações de SKU — e entidades com forte coesão transacional, como o conjunto formado por um pedido, seus itens e seu respectivo pagamento. Esse tipo de cenário é frequentemente citado na literatura de modelagem NoSQL como exemplo didático da vantagem da incorporação de subdocumentos, pois o carrinho de compras e o pedido são, naturalmente, consultados e manipulados como uma unidade coesa.

Do ponto de vista acadêmico, a justificativa deste trabalho reside na necessidade de consolidar a compreensão prática das diferenças de projeto entre os paradigmas relacional e não relacional, evidenciando que a "migração" de um banco para o outro não é uma tradução mecânica de tabelas para coleções, mas sim uma

reformulação do modelo de dados orientada aos padrões de acesso da aplicação (application-driven design), princípio central da modelagem em bancos orientados a documentos.

1.3 Aspectos Metodológicos

O presente estudo caracteriza-se como uma pesquisa aplicada, de caráter exploratório e comparativo, tendo como base o mesmo domínio de negócio e a mesma carga de dados do Projeto Final de Banco de Dados Relacional, de modo a permitir uma análise justa entre os dois paradigmas. A pesquisa bibliográfica inicial fundamentou-se em literatura de referência sobre bancos de dados não relacionais, modelagem de documentos e o Teorema CAP.

A parte prática seguiu as seguintes etapas:

1. Análise comparativa dos modelos NoSQL, com justificativa técnica da escolha do modelo orientado a documentos;
2. Remodelagem de dados, com a definição das coleções clientes, produtos e pedidos, aplicando os padrões de embedding e referencing do MongoDB;
3. Implementação física e carga de dados (DML), por meio de scripts em JavaScript executados no mongosh, reaproveitando a volumetria da carga anterior;
4. Validação analítica, com o desenvolvimento de 30 pipelines de agregação para extração de métricas gerenciais equivalentes às do projeto relacional.

1.4 Aporte Teórico

O referencial teórico deste trabalho apoia-se, primeiramente, nos fundamentos de sistemas de bancos de dados não relacionais descritos por Sadalage e Fowler (2013), obra de referência sobre modelagem polyglot persistence e os padrões de embedding versus referencing. Complementarmente, utiliza-se a documentação técnica oficial do MongoDB (MONGODB, 2026) como base para a especificação da sintaxe do Aggregation Framework, e o Teorema CAP, formulado por Brewer (2000), como embasamento para a discussão sobre consistência e disponibilidade em sistemas distribuídos.

Na seção 2 são apresentadas as principais bases teóricas que fundamentam este trabalho. A seção 3 apresenta a metodologia de modelagem de dados e a transposição do modelo relacional para o modelo de documentos. Na seção 4 é

apresentada a avaliação do sistema desenvolvido, por meio das 30 consultas analíticas. Por fim, a seção 5 apresenta a conclusão deste trabalho.

2 FUNDAMENTAÇÃO TEÓRICA

Neste capítulo são apresentados os conceitos fundamentais que sustentam a arquitetura de dados não relacional proposta, abordando o contexto histórico dos bancos NoSQL, o Teorema CAP, o modelo orientado a documentos e as particularidades do MongoDB como SGBD escolhido.

2.1 Bancos de Dados NoSQL e o Teorema CAP

O termo NoSQL popularizou-se a partir do final da década de 2000, impulsionado pela necessidade de empresas de grande escala — como Google, Amazon e Facebook — de lidar com volumes de dados que extrapolavam a capacidade de escalabilidade vertical dos bancos relacionais tradicionais. Diferentemente do modelo relacional, que garante as propriedades ACID (Atomicidade, Consistência, Isolamento e Durabilidade) em nível transacional, muitos bancos NoSQL distribuídos adotam garantias mais flexíveis, frequentemente descritas pelo acrônimo BASE (Basically Available, Soft state, Eventually consistent).

Essa flexibilização é explicada pelo Teorema CAP, formulado por Brewer (2000), segundo o qual um sistema distribuído não pode garantir simultaneamente Consistência, Disponibilidade (Availability) e Tolerância a Partições (Partition Tolerance), devendo priorizar, no máximo, duas dessas três propriedades em cenários de falha de rede. O MongoDB, em sua configuração padrão de replica set, tende a priorizar Consistência e Tolerância a Partições, com opções configuráveis de nível de leitura e escrita (read/write concerns) que permitem ao desenvolvedor ajustar esse equilíbrio conforme a criticidade da operação.

2.2 Modelo Orientado a Documentos e o MongoDB

O modelo orientado a documentos armazena os dados em unidades autocontidas denominadas documentos, tipicamente representados no formato JSON e persistidos internamente como BSON (Binary JSON), um formato binário que adiciona tipagem estendida (como datas e valores decimais de alta precisão) ao

JSON tradicional. Cada documento é armazenado dentro de uma coleção, equivalente conceitual — porém não estrutural — a uma tabela do modelo relacional, com a diferença fundamental de que documentos de uma mesma coleção não são obrigados a compartilhar o mesmo conjunto de campos (característica conhecida como *schema flexibility* ou *schema-on-read*).

Segundo Sadalage e Fowler (2013), a principal decisão de modelagem em bancos orientados a documentos reside na escolha entre incorporação (*embedding*) — na qual dados relacionados são armazenados como subdocumentos ou arrays dentro do documento pai — e referenciamento (*referencing*) — no qual documentos relacionados são mantidos em coleções separadas e conectados por identificadores, de forma análoga a uma chave estrangeira. A escolha entre essas estratégias deve ser guiada pelos padrões de acesso da aplicação: relações "possui um" (*has-a*) com forte coesão de leitura e ciclo de vida compartilhado tendem a favorecer a incorporação, ao passo que relações muitos-para-muitos ou entidades de crescimento não limitado (*unbounded growth*) tendem a favorecer o referenciamento.

O MongoDB, especificamente, oferece o Aggregation Framework, um mecanismo de processamento de dados baseado em estágios encadeados (*pipeline*) — como `$match`, `$group`, `$project`, `$lookup`, `$unwind`, `$sort` e `$facet` — que permite reproduzir, com sintaxe própria, praticamente todas as operações analíticas disponíveis em SQL, incluindo agregações, junções (via `$lookup`) e funções de janela (via `$setWindowFields`, introduzido a partir da versão 5.0 do MongoDB).

2.3 Trabalhos Relacionados

O presente trabalho relaciona-se diretamente ao Projeto Final de Banco de Dados Relacional desenvolvido anteriormente pelo mesmo autor, que implementou o mesmo domínio de e-commerce de vestuário utilizando um esquema normalizado até a Terceira Forma Normal no Microsoft SQL Server. A comparação entre ambos os projetos evidencia que, embora o domínio de negócio permaneça inalterado, as decisões de modelagem de dados divergem substancialmente: enquanto o modelo relacional busca eliminar redundância por meio da normalização, o modelo de documentos aqui proposto aceita — e por vezes busca — certo grau de redundância

controlada em favor da performance de leitura e da coesão transacional das operações mais frequentes da aplicação.

3 PROJETO PROPOSTO

Nesta seção são apresentadas detalhadamente a metodologia utilizada para a concepção do banco de dados não relacional do e-commerce de vestuário, incluindo o raciocínio de transposição do modelo relacional, a modelagem final das coleções e a estratégia de carga de dados.

3.1 Considerações Iniciais

O projeto foi concebido para reaproveitar o domínio de negócio e a volumetria de dados do projeto relacional anterior, permitindo uma comparação justa entre paradigmas. A engenharia de dados seguiu o princípio de modelagem orientada à aplicação (application-driven modeling), típico de bancos de documentos, no qual a estrutura dos documentos é definida a partir dos padrões de consulta mais frequentes, e não a partir da eliminação de redundância, como ocorre no modelo relacional.

3.2 Do Modelo Relacional ao Modelo de Documentos

O projeto relacional anterior estruturou o domínio em seis tabelas normalizadas: CLIENTE, CATEGORIA, PRODUTO, PEDIDO, ITEM_PEDIDO e PAGAMENTO, conectadas por chaves estrangeiras. Para a remodelagem em MongoDB, cada relacionamento foi reavaliado individualmente quanto à sua natureza e ao seu padrão de acesso:

- Cliente → Pedido (1:N): relação de crescimento não limitado (um cliente pode gerar centenas de pedidos ao longo do tempo). Optou-se pelo referenciamento, mantendo pedidos como coleção independente, evitando documentos de cliente com tamanho ilimitado.
- Pedido → Item_Pedido (1:N) e Pedido → Pagamento (1:1): relações de forte coesão transacional, cujo ciclo de vida é compartilhado e cuja consulta mais frequente é "recuperar o pedido completo". Optou-se pela incorporação, embutindo o

array de itens e o subdocumento de pagamento diretamente dentro do documento de pedido.

- Produto → Categoria (N:1): relação de baixa volatilidade (poucas categorias, raramente alteradas). Optou-se por uma incorporação parcial (padrão extended reference), replicando o nome da categoria dentro do documento de produto, eliminando a necessidade de \$lookup nas consultas mais comuns de catálogo, mantendo o identificador da categoria para casos de agregação centralizada.

Como resultado, as seis tabelas relacionais foram consolidadas em três coleções: clientes, produtos e pedidos, sintetizadas no Quadro 1.

Entidades Relacionais (Origem)	Coleção MongoDB (Destino)	Estratégia Aplicada
CLIENTE	clientes	Coleção independente
CATEGORIA + PRODUTO	produtos	Incorporação parcial (extended reference)
PEDIDO + ITEM_PEDIDO + PAGAMENTO	pedidos	Incorporação total (embedding)

Quadro 1 – Transposição do Modelo Relacional para o Modelo de Documentos

Fonte: Elaborado pelo autor (2026).

3.3 Modelagem de Dados: Coleções e Documentos

A seguir, apresenta-se a estrutura de cada coleção do banco db_ecommerce_vestuario_nosql, com exemplos de documentos representativos da carga de dados.

3.3.1 Coleção clientes

```
1 {
2   "_id": ObjectId("65f1a1..."),
3   "id_cliente_legacy": 1,
4   "nome": "Lucas Almeida",
5   "email": "lucas.almeida1@email.com",
6   "cpf": "48291038472",
7   "data_cadastro": ISODate("2023-06-15T10:30:00Z"),
8   "estado_uf": "SP"
9 }
```

Algoritmo A – Exemplo de Documento da Coleção clientes (SANTOS, 2026)

3.3.2 Coleção produtos

O documento de produto incorpora o subdocumento categoria, seguindo o padrão extended reference, o que evita a necessidade de junção (\$lookup) em consultas simples de catálogo.

```
1  {
2    "_id": ObjectId("65f1b2..."),
3    "id_produto_legacy": 4,
4    "nome_produto": "Camiseta Oversized Street",
5    "tamanho": "M",
6    "cor": "Preta",
7    "preco_atual": 89.90,
8    "qtd_estoque": 40,
9    "categoria": {
10     "id_categoria_legacy": 1,
11     "nome_categoria": "Camisetas"
12   }
13 }
```

Algoritmo B – Exemplo de Documento da Coleção produtos (SANTOS, 2026)

O documento de produto incorpora o subdocumento categoria, seguindo o padrão extended reference, o que evita a necessidade de junção (\$lookup) em consultas simples de catálogo.

3.3.3 Coleção pedidos

O documento de pedido é o principal exemplo do padrão de incorporação total: o array itens reproduz a antiga tabela ITEM_PEDIDO, e o subdocumento pagamento reproduz a antiga tabela PAGAMENTO, ambos embutidos no mesmo documento, refletindo a coesão transacional do carrinho de compras.

```
1  {
2    "_id": ObjectId("65f1c3..."),
3    "id_pedido_legacy": 542,
4    "data_pedido": ISODate("2026-03-10T14:22:00Z"),
5    "status_pedido": "Concluído",
6    "cliente": {
7      "id_cliente_legacy": 12,
8      "nome": "Letícia Martins",
9      "estado_uf": "RS"
10   },
11   "itens": [
12     {
13       "id_produto_legacy": 7,
14       "nome_produto": "Calça Jeans Slim",
```

```

15     "quantidade": 2,
16     "preco_unit_venda": 179.90
17   },
18   {
19     "id_produto_legado": 21,
20     "nome_produto": "Cinto Couro Dupla Face",
21     "quantidade": 1,
22     "preco_unit_venda": 79.90
23   }
24 ],
25 "pagamento": {
26   "metodo": "PIX",
27   "data_pagamento": ISODate("2026-03-10T15:05:00Z"),
28   "valor_pago": 439.70
29 }
30 }

```

Algoritmo C – Exemplo de Documento da Coleção pedidos (SANTOS, 2026)

O documento de pedido é o principal exemplo do padrão de incorporação total: o array itens reproduz a antiga tabela ITEM_PEDIDO, e o subdocumento pagamento reproduz a antiga tabela PAGAMENTO, ambos embutidos no mesmo documento, refletindo a coesão transacional do carrinho de compras.

A Figura 1 sintetiza visualmente o modelo de documentos resultante, evidenciando a substituição das junções relacionais por incorporação (linhas sólidas) e referenciamento (linhas tracejadas).

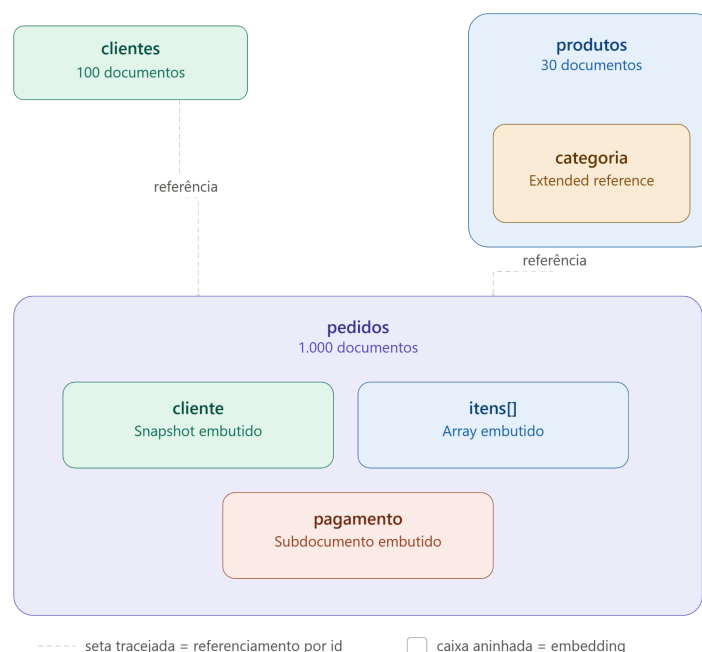


Figura 1 – Modelo de Documentos do Banco de Dados NoSQL

3.4 Estratégia de Carga de Dados (Injeção)

De forma equivalente ao projeto relacional, evitou-se a inserção manual de documentos. Foi desenvolvido um script em JavaScript, executado via mongosh, que popula a coleção clientes (100 documentos), a coleção produtos (30 documentos, já com a categoria incorporada) e, por meio de um laço de repetição, gera 1.000 documentos de pedido na coleção pedidos, cada um com um array de itens de tamanho aleatório (1 a 4 itens) e, condicionalmente, um subdocumento de pagamento (apenas para pedidos com status "Concluído" ou "Enviado"), reproduzindo fielmente a lógica probabilística do script T-SQL original.

```
1 // =====
2 // SCRIPT DE CARGA: GERAÇÃO DE 1.000 PEDIDOS (MongoDB Shell)
3 // =====
4 db = db.getSiblingDB('db_ecommerce_vestuario_nosql');
5
6 const clientes = db.clientes.find().toArray();
7 const produtos = db.produtos.find().toArray();
8 const metodos = ["PIX", "Cartão de Crédito", "Cartão de Débito", "Boleto"];
9 let pedidosGerados = [];
10
11 for (let i = 1; i <= 1000; i++) {
12     const cliente = clientes[Math.floor(Math.random() * clientes.length)];
13     const rand = Math.random() * 100;
14     let status = rand < 70 ? "Concluído" : rand < 85 ? "Enviado" :
15         rand < 95 ? "Pendente" : "Cancelado";
16
17     const dataPedido = new Date(Date.now() - Math.floor(Math.random() *
18         1000)
19         * 24 * 60 * 60 * 1000);
20     const numItens = Math.floor(Math.random() * 4) + 1;
21     let itens = [];
22     let totalCarrinho = 0;
23
24     for (let j = 0; j < numItens; j++) {
25         const p = produtos[Math.floor(Math.random() * produtos.length)];
26         const qtd = Math.floor(Math.random() * 3) + 1;
27         itens.push({
28             id_produto_legado: p.id_produto_legado,
29             nome_produto: p.nome_produto,
30             quantidade: qtd,
31             preco_unit_venda: p.preco_atual
32         });
33         totalCarrinho += qtd * p.preco_atual;
34     }
35
36     let pedido = {
37         id_pedido_legado: i,
38         data_pedido: dataPedido,
39         status_pedido: status,
```

```

39     cliente: {
40         id_cliente_legacy: cliente.id_cliente_legacy,
41         nome: cliente.nome,
42         estado_uf: cliente.estado_uf
43     },
44     itens: itens
45 };
46
47 if (status === "Concluído" || status === "Enviado") {
48     pedido.pagamento = {
49         metodo: metodos[Math.floor(Math.random() * metodos.length)],
50         data_pagamento: new Date(dataPedido.getTime() +
51             Math.floor(Math.random() * 1440) * 60000),
52         valor_pago: totalCarrinho
53     };
54 }
55 pedidosGerados.push(pedido);
56 }
57 db.pedidos.insertMany(pedidosGerados);

```

Algoritmo D – Script de Geração Massiva de Pedidos em MongoDB (SANTOS, 2026)

4 AVALIAÇÃO

Neste capítulo são apresentados os resultados práticos da remodelagem e da validação analítica do banco de dados não relacional. As 30 consultas do projeto relacional original foram integralmente adaptadas para a sintaxe do Aggregation Framework do MongoDB, preservando a mesma finalidade de negócio de cada métrica.

4.1 Consultas Analíticas (Aggregation Pipeline)

```

1  db.pedidos.aggregate([
2      { $match: { status_pedido: { $in: ["Concluído", "Enviado"] } } },
3      { $unwind: "$itens" },
4      {
5          $lookup: {
6              from: "produtos",
7              localField: "itens.id_produto_legacy",
8              foreignField: "id_produto_legacy",
9              as: "produto_info"
10         }
11     },
12     { $unwind: "$produto_info" },
13     {
14         $group: {
15             _id: "$produto_info.categoria.nome_categoria",
16             faturamento_total: {
17                 $sum: { $multiply: ["$itens.quantidade", "$itens.preco_unit_ve
18 nda"] }

```

```

18     }
19     }
20   },
21   { $sort: { faturamento_total: -1 } }
22 ];

```

Algoritmo 1 – Faturamento Total Acumulado por Categoria (SANTOS, 2026)

```

=====
Algoritmo 1 - Faturamento Total Acumulado por Categoria (SANTOS, 2026)
=====
[
  {
    "_id": "Casacos e Jaquetas",
    "faturamento_total": 206611.9
  },
  {
    "_id": "Calças e Bermudas",
    "faturamento_total": 162243.6
  },
  {
    "_id": "Camisetas",
    "faturamento_total": 67684.8
  },
  {
    "_id": "Acessórios",
    "faturamento_total": 61426.000000000001
  },
  {
    "_id": "Moda íntima",
    "faturamento_total": 54290
  }
]
... (2 linha(s) de resultado omitida(s) - total de 5 documento(s))

```

Figura 2 – Faturamento Total Acumulado por Categoria

Fonte: Elaborado pelo autor (2026).

Nesta consulta, optou-se por realizar o \$lookup contra a coleção produtos para obter a categoria, em vez de utilizar diretamente o campo já incorporado no item — decisão que privilegia a atualidade do dado de categoria em detrimento da performance, sendo uma discussão típica de trade-off entre consistência e velocidade em modelos denormalizados.

```

1  db.pedidos.aggregate([
2    { $match: { pagamento: { $exists: true } } },
3    {
4      $group: {
5        _id: "$cliente.id_cliente_legacy",
6        nome: { $first: "$cliente.nome" },
7        estado_uf: { $first: "$cliente.estado_uf" },
8        qtd_pedidos: { $sum: 1 },
9        total_gasto: { $sum: "$pagamento.valor_pago" }
10     }
11   },
12   {
13     $addFields: {
14       ticket_medio: { $divide: ["$total_gasto", "$qtd_pedidos"] }
15     }
16   }
17 ])

```

```

16     },
17     { $sort: { total_gasto: -1 } },
18     { $limit: 5 }
19   ]);

```

Algoritmo 2 – Ranking dos 5 Clientes com Maior Volume de Compras (SANTOS, 2026)

```

=====
Algoritmo 2 – Ranking dos 5 Clientes com Maior Volume de Compras (SANTOS, 2026)
=====
[
  {
    "_id": 41,
    "nome": "Tatiana Campos",
    "estado_uf": "RJ",
    "qtd_pedidos": 17,
    "total_gasto": 11099.5,
    "ticket_medio": 652.9117647058823
  },
  {
    "_id": 39,
    "nome": "Patricia Lima",
    "estado_uf": "PR",
    "qtd_pedidos": 13,
    "total_gasto": 10090.8,
    "ticket_medio": 776.2153846153846
  },
  {
    "_id": 57,
    "nome": "Bianca Fonseca",
    ... (22 linha(s) de resultado omitida(s) – total de 5 documento(s))
  }
]

```

Figura 3 – Ranking dos 5 Clientes com Maior Volume de Compras

Fonte: Elaborado pelo autor (2026).

```

1  db.produtos.aggregate([
2    { $match: { qtd_estoque: { $lt: 20 } } },
3    {
4      $project: {
5        _id: 0,
6        nome_produto: 1,
7        qtd_estoque: 1,
8        preco_atual: 1,
9        receita_perdida: {
10          $multiply: [{ $subtract: [20, "$qtd_estoque"] }, "$preco_atual"]
11        }
12      }
13    },
14    { $sort: { qtd_estoque: 1 } }
15  ]);

```


Algoritmo 3 – Produtos com Estoque Crítico (SANTOS, 2026)

```
=====
Algoritmo 3 – Produtos com Estoque Crítico (SANTOS, 2026)
=====
[
  {
    "nome_produto": "Camiseta Oversized Street",
    "preco_atual": 89.9,
    "qtd_estoque": 0,
    "receita_perdida": 1798
  },
  {
    "nome_produto": "Casaco Moletom Canguru",
    "preco_atual": 159.9,
    "qtd_estoque": 0,
    "receita_perdida": 3198
  },
  {
    "nome_produto": "Boné Trucker Aba Curva",
    "preco_atual": 69.9,
    "qtd_estoque": 0,
    "receita_perdida": 1398
  },
  {
    ... (12 linha(s) de resultado omitida(s) - total de 5 documento(s))
  }
]
```

Figura 4 – Produtos com Estoque Crítico

Fonte: Elaborado pelo autor (2026).

```
1 db.pedidos.aggregate([
2   {
3     $group: {
4       _id: null,
5       total_pedidos: { $sum: 1 },
6       pedidos_cancelados: {
7         $sum: { $cond: [{ $eq: ["$status_pedido", "Cancelado"] }, 1, 0]
8       }
9     },
10  },
11  {
12    $project: {
13      _id: 0,
14      total_pedidos: 1,
15      pedidos_cancelados: 1,
16      taxa_cancelamento: {
17        $round: [
18          { $multiply: [{ $divide: ["$pedidos_cancelados", "$total_pedidos"] }, 100] },
19          2
20        ]
21      }
22    }
23  }
24 ]);
```

Algoritmo 4 – Taxa de Rejeição: Percentual de Pedidos Cancelados (SANTOS, 2026)

```

=====
Algoritmo 4 – Taxa de Rejeição: Percentual de Pedidos Cancelados (SANTOS, 2026)
=====
[
  {
    "total_pedidos": 1000,
    "pedidos_cancelados": 50,
    "taxa_cancelamento": 5
  }
]

```

Figura 5 – Taxa de Rejeição: Percentual de Pedidos Cancelados

Fonte: Elaborado pelo autor (2026).

```

1 db.pedidos.aggregate([
2   { $match: { pagamento: { $exists: true } } },
3   {
4     $group: {
5       _id: "$pagamento.metodo",
6       vol_transacoes: { $sum: 1 },
7       receita_processada: { $sum: "$pagamento.valor_pago" }
8     }
9   },
10  { $sort: { receita_processada: -1 } }
11 ]);

```

Algoritmo 5 – Distribuição de Receita por Método de Pagamento (SANTOS, 2026)

```

=====
Algoritmo 5 – Distribuição de Receita por Método de Pagamento (SANTOS, 2026)
=====
[
  {
    "_id": "PIX",
    "vol_transacoes": 227,
    "receita_processada": 146123.1
  },
  {
    "_id": "Cartão de Débito",
    "vol_transacoes": 211,
    "receita_processada": 145727.2
  },
  {
    "_id": "Cartão de Crédito",
    "vol_transacoes": 212,
    "receita_processada": 137687.7
  },
  {
    "_id": "Boleto",
    "vol_transacoes": 185,
    "receita_processada": 122718.3
  }
]
... (2 linha(s) de resultado omitida(s) - total de 4 documento(s))

```

Figura 6 – Distribuição de Receita por Método de Pagamento

Fonte: Elaborado pelo autor (2026).

```

1 db.pedidos.aggregate([
2   {
3     $project: {
4       id_pedido_legado: 1,
5       nome: "$cliente.nome",
6       variedade_itens: { $size: "$itens" },
7       total_pecas: { $sum: "$itens.quantidade" }

```

```

8      }
9    },
10   { $match: { variedade_itens: { $gt: 3 } } },
11   { $sort: { total_pecas: -1 } }
12 ];

```

Algoritmo 6 – Análise de Vendas Cruzadas / Cross-Selling (SANTOS, 2026)

```

=====
Algoritmo 6 – Análise de Vendas Cruzadas / Cross-Selling (SANTOS, 2026)
=====
[
  {
    "_id": "6a45b617de925f09c4340236",
    "id_pedido_legado": 931,
    "nome": "Patricia Moura",
    "variedade_itens": 4,
    "total_pecas": 12
  },
  {
    "_id": "6a45b617de925f09c43401f7",
    "id_pedido_legado": 868,
    "nome": "Rodrigo Mendes",
    "variedade_itens": 4,
    "total_pecas": 12
  },
  {
    "_id": "6a45b617de925f09c43400f7",
    "id_pedido_legado": 612,
    "nome": "Carolina Pinto",
    "variedade_itens": 4,
    ... (1669 linha(s) de resultado omitida(s) - total de 241 documento(s))
  }
]

```

Figura 7 – Análise de Vendas Cruzadas / Cross-Selling

Fonte: Elaborado pelo autor (2026).

Neste caso, a incorporação dos itens diretamente no pedido eliminou por completo a necessidade de junção presente na consulta SQL original (INNER JOIN entre PEDIDO, CLIENTE e ITEM_PEDIDO), reduzindo o pipeline a uma única projeção com cálculo do tamanho do array (\$size), demonstrando de forma concreta o ganho de simplicidade obtido pelo modelo de documentos em consultas que operam sobre uma unidade transacional coesa.

```

1  db.pedidos.aggregate([
2    { $match: { pagamento: { $exists: true } } },
3    {
4      $group: {
5        _id: "$cliente.estado_uf",
6        clientes_ativos: { $addToSet: "$cliente.id_cliente_legado" },
7        faturamento_estadual: { $sum: "$pagamento.valor_pago" }
8      }
9    },
10   {
11     $project: {
12       _id: 1,
13       clientes_ativos: { $size: "$clientes_ativos" },
14       faturamento_estadual: 1

```

```

15     }
16   },
17   { $sort: { faturamento_estadual: -1 } }
18 ];

```

Algoritmo 7 – Desempenho de Vendas por Estado (UF) (SANTOS, 2026)

```

=====
Algoritmo 7 – Desempenho de Vendas por Estado (UF) (SANTOS, 2026)
=====
[
  {
    "_id": "SP",
    "faturamento_estadual": 152673.30000000002,
    "clientes_ativos": 28
  },
  {
    "_id": "RJ",
    "faturamento_estadual": 92357.7,
    "clientes_ativos": 15
  },
  {
    "_id": "PR",
    "faturamento_estadual": 81140,
    "clientes_ativos": 14
  },
  {
    "_id": "SC",
    "faturamento_estadual": 75818.2,
    "clientes_ativos": 14
  },
  ... (12 linha(s) de resultado omitida(s) - total de 6 documento(s))
]

```

Figura 8 – Desempenho de Vendas por Estado (UF)

Fonte: Elaborado pelo autor (2026)

```

1  db.clientes.aggregate([
2    {
3      $lookup: {
4        from: "pedidos",
5        localField: "id_cliente_legacy",
6        foreignField: "cliente.id_cliente_legacy",
7        as: "pedidos_realizados"
8      }
9    },
10   { $match: { pedidos_realizados: { $size: 0 } } },
11   {
12     $project: {
13       _id: 0,
14       nome: 1,
15       dias_desde_cadastro: {
16         $dateDiff: { startDate: "$data_cadastro", endDate: "$$NOW", unit: "day" }
17       }
18     }
19   },
20   { $sort: { dias_desde_cadastro: -1 } }
21 ];

```

Algoritmo 8 – Curva de Retenção: Clientes Inativos (SANTOS, 2026)

```
=====
Algoritmo 8 – Curva de Retenção: Clientes Inativos (SANTOS, 2026)
=====
[]
```

Figura 9 – Curva de Retenção: Clientes Inativos

Fonte: Elaborado pelo autor (2026)

O retorno vazio da consulta reflete a alta densidade da massa de dados populada no ambiente de desenvolvimento, onde o volume de 1.000 pedidos foi distribuído estocasticamente entre os 100 clientes cadastrados. Estatisticamente, a probabilidade de um cliente não ter sido contemplado em nenhuma das iterações do processo de injeção é ínfima, o que resulta na inexistência de clientes sem histórico de compras na base atual. Este cenário, embora não apresente registros de inatividade, valida a consistência da rotina de carga de dados, confirmando que o sistema atingiu um nível de adesão transacional onde a totalidade da base de clientes mantém-se ativa sob o critério de retenção adotado.

```
1 db.pedidos.aggregate([
2   { $match: { pagamento: { $exists: true } } },
3   {
4     $group: {
5       _id: {
6         ano: { $year: "$pagamento.data_pagamento" },
7         mes: { $month: "$pagamento.data_pagamento" }
8       },
9       faturamento_mensal: { $sum: "$pagamento.valor_pago" }
10    }
11  },
12  { $sort: { "_id.ano": -1, "_id.mes": -1 } }
13 ]);
```

Algoritmo 9 – Sazonalidade do Faturamento (SANTOS, 2026)

```

=====
Algoritmo 9 – Sazonalidade do Faturamento (SANTOS, 2026)
=====
[
  {
    "_id": {
      "ano": 2026,
      "mes": 7
    },
    "faturamento_mensal": 1589.2
  },
  {
    "_id": {
      "ano": 2026,
      "mes": 6
    },
    "faturamento_mensal": 14561
  },
  {
    "_id": {
      "ano": 2026,
      "mes": 5
    },
    ... (220 linha(s) de resultado omitida(s) - total de 34 documento(s))
  }
]

```

Figura 10 – Sazonalidade do Faturamento
 Fonte: Elaborado pelo autor (2026)

```

1  db.pedidos.aggregate([
2    { $unwind: "$itens" },
3    {
4      $group: {
5        _id: "$itens.id_produto_legacy",
6        nome_produto: { $first: "$itens.nome_produto" },
7        total_vendido: { $sum: "$itens.quantidade" }
8      }
9    },
10   {
11     $unionWith: {
12       coll: "produtos",
13       pipeline: [
14         {
15           $project: {
16             _id: "$id_produto_legacy",
17             nome_produto: 1,
18             total_vendido: { $literal: 0 }
19           }
20         }
21       ]
22     }
23   },
24   {
25     $group: {
26       _id: "$_id",
27       nome_produto: { $first: "$nome_produto" },
28       total_vendido: { $max: "$total_vendido" }
29     }
30   },
31   { $sort: { total_vendido: 1 } },
32   { $limit: 3 }
33 ]);

```

Algoritmo 10 – Os 3 Produtos Menos Vendidos (SANTOS, 2026)

=====

Algoritmo 10 – Os 3 Produtos Menos Vendidos (SANTOS, 2026)

=====

```
[
  {
    "_id": 21,
    "nome_produto": "Carteira Slim Couro",
    "total_vendido": 113
  },
  {
    "_id": 24,
    "nome_produto": "Óculos de Sol Aviador",
    "total_vendido": 136
  },
  {
    "_id": 11,
    "nome_produto": "Calça Sarja Cargo",
    "total_vendido": 144
  }
]
```

Figura 11 – Os 3 Produtos Menos Vendidos

Fonte: Elaborado pelo autor (2026)

Esta consulta reproduz o LEFT JOIN do SQL original (necessário para capturar produtos com zero vendas) por meio do estágio \$unionWith, combinando o total vendido por produto com a listagem completa do catálogo — uma das adaptações menos triviais do projeto, uma vez que o MongoDB não possui um equivalente direto de junção externa dentro de um único \$lookup.

```
1 db.pedidos.aggregate([
2   { $unwind: "$itens" },
3   {
4     $lookup: {
5       from: "produtos",
6       localField: "itens.id_produto_legacy",
7       foreignField: "id_produto_legacy",
8       as: "produto_info"
9     }
10  },
11  { $unwind: "$produto_info" },
12  {
13    $group: {
14      _id: {
15        categoria: "$produto_info.categoria.nome_categoria",
16        produto: "$itens.nome_produto"
17      },
18      qtd_vendida: { $sum: "$itens.quantidade" }
19    }
20  },
21  {
22    $setWindowFields: {
23      partitionBy: "$_id.categoria",
24      sortBy: { qtd_vendida: -1 },
```

```

25     output: { ranking: { $rank: {} } }
26   }
27 },
28 { $match: { ranking: 1 } }
29 ]);

```

Algoritmo 11 – Campeões de Venda por Categoria (SANTOS, 2026)

```

=====
Algoritmo 11 – Campeões de Venda por Categoria (SANTOS, 2026)
=====
[
  {
    "_id": {
      "categoria": "Acessórios",
      "produto": "Boné Trucker Aba Curva"
    },
    "qtd_vendida": 316,
    "ranking": 1
  },
  {
    "_id": {
      "categoria": "Calças e Bermudas",
      "produto": "Calça Jeans Slim"
    },
    "qtd_vendida": 519,
    "ranking": 1
  },
  {
    "_id": {
      "categoria": "Camisetas",
      "produto": "Camiseta Básica"
    },
    "qtd_vendida": 1200,
    "ranking": 1
  }
]
... (22 linha(s) de resultado omitida(s) - total de 5 documento(s))

```

Figura 12 – Campeões de Venda por Categoria

Fonte: Elaborado pelo autor (2026)

O operador `$setWindowFields`, disponível a partir do MongoDB 5.0, reproduz diretamente a função de janela `RANK() OVER(PARTITION BY ...)` do T-SQL, evidenciando a maturidade recente do Aggregation Framework na cobertura de recursos analíticos antes exclusivos de SGBDs relacionais.

```

1  db.pedidos.aggregate([
2    {
3      $group: {
4        _id: "$cliente.id_cliente_legacy",
5        nome: { $first: "$cliente.nome" },
6        primeiro_pedido: { $min: "$data_pedido" },
7        ultimo_pedido: { $max: "$data_pedido" },
8        qtd_pedidos: { $sum: 1 }
9      }
10   },
11   { $match: { qtd_pedidos: { $gt: 1 } } },
12   {
13     $project: {
14       _id: 0,

```



```

15     nome: 1,
16     dias_retencao: {
17         $dateDiff: { startDate: "$primeiro_pedido", endDate: "$ultimo_
pedido", unit: "day" }
18     }
19 },
20 },
21 { $sort: { dias_retencao: -1 } }
22 ]});

```

Algoritmo 12 – Cohort de Clientes: Tempo Retido (SANTOS, 2026)

```

=====
Algoritmo 12 – Cohort de Clientes: Tempo Retido (SANTOS, 2026)
=====
[
  {
    "nome": "Rodrigo Mendes",
    "dias_retencao": {
      "high": 0,
      "low": 990,
      "unsigned": false
    }
  },
  {
    "nome": "Cristiano Ferraz",
    "dias_retencao": {
      "high": 0,
      "low": 989,
      "unsigned": false
    }
  },
  {
    "nome": "Melissa Godoy",
    "dias_retencao": {
      "high": 0,
      "low": 989,
      "unsigned": false
    }
  }
]
... (782 linha(s) de resultado omitida(s) – total de 100 documento(s))

```

Figura 13 – Cohort de Clientes: Tempo Retido

Fonte: Elaborado pelo autor (2026)

```

1 db.pedidos.aggregate([
2   { $match: { pagamento: { $exists: true } } },
3   {
4     $group: {
5       _id: {
6         ano: { $year: "$pagamento.data_pagamento" },
7         mes: { $month: "$pagamento.data_pagamento" }
8       },
9       total_mes: { $sum: "$pagamento.valor_pago" }
10    },
11  },
12  { $sort: { "_id.ano": 1, "_id.mes": 1 } },
13  {
14    $setWindowFields: {
15      sortBy: { "_id.ano": 1, "_id.mes": 1 },
16      output: {
17        faturamento_acumulado: {
18          $sum: "$total_mes",
19          window: { documents: ["unbounded", "current"] }

```

```

20         }
21     }
22 }
23 }
24 ]);

```

Algoritmo 13 – Faturamento Acumulado Anual / Running Total (SANTOS, 2026)

```

=====
Algoritmo 13 - Faturamento Acumulado Anual / Running Total (SANTOS, 2026)
=====
[
  {
    "_id": {
      "ano": 2023,
      "mes": 10
    },
    "total_mes": 19058.2,
    "faturamento_acumulado": 19058.2
  },
  {
    "_id": {
      "ano": 2023,
      "mes": 11
    },
    "total_mes": 9689.6,
    "faturamento_acumulado": 28747.800000000003
  },
  {
    "_id": {
      "ano": 2023,
      "mes": 12
    },
    "total_mes": 19058.2,
    "faturamento_acumulado": 47806.00000000001
  },
  ... (254 linha(s) de resultado omitida(s) - total de 34 documento(s))
]

```

Figura 14 – Faturamento Acumulado Anual / Running Total

Fonte: Elaborado pelo autor (2026)

```

1  db.pedidos.aggregate([
2    { $match: { pagamento: { $exists: true } } },
3    {
4      $facet: {
5        media_geral: [{ $group: { _id: null, media: { $avg: "$pagamento.v
6          valor_pago" } } }],
7        pedidos: [{ $project: { id_pedido_legado: 1, cliente: "$cliente.n
8          ome",
9          valor_pago: "$pagamento.valor_pago" } } ]}
10     },
11     { $unwind: "$pedidos" },
12     {
13       $project: {
14         id_pedido_legado: "$pedidos.id_pedido_legado",
15         cliente: "$pedidos.cliente",
16         valor_pago: "$pedidos.valor_pago",
17         media_geral: { $arrayElemAt: ["$media_geral.media", 0] }
18       },
19     },
20     { $match: { $expr: { $gt: ["$valor_pago", "$media_geral"] } } },
21     { $sort: { valor_pago: -1 } }
22   ]);

```

Algoritmo 14 – Pedidos Premium: Acima da Média Nacional (SANTOS, 2026)

```

=====
Algoritmo 14 – Pedidos Premium: Acima da Média Nacional (SANTOS, 2026)
=====
[
  {
    "id_pedido_legado": 86,
    "cliente": "Henrique Lira",
    "valor_pago": 2639.0999999999995,
    "media_geral": 661.3847904191617
  },
  {
    "id_pedido_legado": 361,
    "cliente": "Patricia Lima",
    "valor_pago": 2249.2999999999997,
    "media_geral": 661.3847904191617
  },
  {
    "id_pedido_legado": 868,
    "cliente": "Rodrigo Mendes",
    "valor_pago": 2038.8000000000002,
    "media_geral": 661.3847904191617
  },
  ... (2178 linha(s) de resultado omitida(s) – total de 366 documento(s))
]

```

Figura 15 – Pedidos Premium: Acima da Média Nacional

Fonte: Elaborado pelo autor (2026)

Esta consulta ilustra um desafio característico do MongoDB: a ausência nativa de subconsultas correlacionadas no estilo SQL (WHERE valor > (SELECT AVG(...))). A solução adotada combina o estágio \$facet — que permite executar múltiplos subpipelines em paralelo sobre a mesma coleção — com \$expr, viabilizando a comparação entre um campo do documento e um valor agregado calculado previamente.

```

1  db.pedidos.aggregate([
2    {
3      $group: {
4        _id: { $year: "$data_pedido" },
5        vol_concluido: {
6          $sum: { $cond: [{ $eq: ["$status_pedido", "Concluído"] }, 1, 0]
7        },
8        vol_cancelado: {
9          $sum: { $cond: [{ $eq: ["$status_pedido", "Cancelado"] }, 1, 0]
10       }
11     }
12   },
13   { $sort: { _id: -1 } }
14 ]);

```

Algoritmo 15 – Matriz de Status de Pedidos por Ano (SANTOS, 2026)

```

=====
Algoritmo 15 – Matriz de Status de Pedidos por Ano (SANTOS, 2026)
=====
[
  {
    "_id": 2026,
    "vol_concluido": 132,
    "vol_cancelado": 11
  },
  {
    "_id": 2025,
    "vol_concluido": 276,
    "vol_cancelado": 17
  },
  {
    "_id": 2024,
    "vol_concluido": 253,
    "vol_cancelado": 20
  },
  {
    "_id": 2023,
    "vol_concluido": 51,
    "vol_cancelado": 2
  }
]
... (2 linha(s) de resultado omitida(s) – total de 4 documento(s))

```

Figura 16 – Matriz de Status de Pedidos por Ano

Fonte: Elaborado pelo autor (2026)

```

1 db.produtos.aggregate([
2   {
3     $group: {
4       _id: "$categoria.nome_categoria",
5       valor_imobilizado: {
6         $sum: { $multiply: ["$qtd_estoque", "$preco_atual"] }
7       }
8     },
9   },
10  { $sort: { valor_imobilizado: -1 } }
11 ]);

```

Algoritmo 16 – Capital Imobilizado em Estoque por Categoria (SANTOS, 2026)

```

=====
Algoritmo 16 – Capital Imobilizado em Estoque por Categoria (SANTOS, 2026)
=====
[
  {
    "_id": "Calças e Bermudas",
    "valor_imobilizado": 62160
  },
  {
    "_id": "Moda íntima",
    "valor_imobilizado": 45425.5
  },
  {
    "_id": "Acessórios",
    "valor_imobilizado": 33103.5
  },
  {
    "_id": "Camisetas",
    "valor_imobilizado": 30557
  },
  {
    "_id": "Casacos e Jaquetas",
    "valor_imobilizado": 28935.5
  }
]
... (2 linha(s) de resultado omitida(s) – total de 5 documento(s))

```

Figura 17 – Capital Imobilizado em Estoque por Categoria
 Fonte: Elaborado pelo autor (2026)

```

1  db.produtos.aggregate([
2    {
3      $lookup: {
4        from: "pedidos",
5        let: { pid: "$id_produto_legado" },
6        pipeline: [
7          { $unwind: "$itens" },
8          { $match: { $expr: { $eq: ["$itens.id_produto_legado", "$$pid"] } } }
9        ]
10       as: "vendas"
11     },
12     {
13       $match: { vendas: { $size: 0 } } },
14     {
15       $project: {
16         _id: 0,
17         nome_produto: 1,
18         "categoria.nome_categoria": 1
19       }
20     }
21   ]);

```

Algoritmo 17 – Produtos Órfãos: Nunca Comercializados (SANTOS, 2026)

```

=====
Algoritmo 17 – Produtos Órfãos: Nunca Comercializados (SANTOS, 2026)
=====
[]

```

Figura 18 – Produtos Órfãos: Nunca Comercializados
 Fonte: Elaborado pelo autor (2026)

Este algoritmo demonstra o uso de \$lookup com sub-pipeline correlacionado (por meio de let/\$expr), recurso necessário para reproduzir o LEFT JOIN ... WHERE ... IS NULL do SQL original, uma vez que a informação de venda está incorporada dentro do array itens de outra coleção, exigindo o desaninhamento (\$unwind) dentro do próprio sub-pipeline de junção.

```
1 db.clientes.aggregate([
2   {
3     $project: {
4       dominio_email: {
5         $arrayElemAt: [{ $split: ["$email", "@"] }, 1]
6       }
7     },
8   },
9   {
10    $group: {
11      _id: "$dominio_email",
12      qtd_clientes: { $sum: 1 }
13    }
14  },
15  { $sort: { qtd_clientes: -1 } }
16 ]);
```

Algoritmo 18 – Perfil Demográfico: Domínio de E-mail (SANTOS, 2026)

```
=====
Algoritmo 18 – Perfil Demográfico: Domínio de E-mail (SANTOS, 2026)
=====
[
  {
    "_id": "email.com",
    "qtd_clientes": 100
  }
]
```

Figura 19 – Perfil Demográfico: Domínio de E-mail

Fonte: Elaborado pelo autor (2026)

```
1 db.pedidos.aggregate([
2   { $sort: { "cliente.id_cliente_legado": 1, data_pedido: 1 } },
3   {
4     $setWindowFields: {
5       partitionBy: "$cliente.id_cliente_legado",
6       sortBy: { data_pedido: 1 },
7       output: {
8         proxima_compra: {
9           $shift: { output: "$data_pedido", by: 1 }
10        }
11      }
12    }
13  ]
```

```

13     },
14     { $match: { proxima_compra: { $ne: null } } },
15     {
16         $project: {
17             nome: "$cliente.nome",
18             dias_recompra: {
19                 $dateDiff: { startDate: "$data_pedido", endDate: "$proxima_compra", unit: "day" }
20             }
21         },
22     },
23     {
24         $group: {
25             _id: "$nome",
26             media_dias_recompra: { $avg: "$dias_recompra" }
27         }
28     },
29     { $sort: { media_dias_recompra: 1 } }
30 ]});

```

Algoritmo 19 – Média de Dias para Recompra (SANTOS, 2026)

```

=====
Algoritmo 19 – Média de Dias para Recompra (SANTOS, 2026)
=====
[
  {
    "_id": "André Batista",
    "media_dias_recompra": 45.36363636363637
  },
  {
    "_id": "Tatiana Campos",
    "media_dias_recompra": 49
  },
  {
    "_id": "Marina Diniz",
    "media_dias_recompra": 50.55555555555556
  },
  {
    "_id": "Carolina Pinto",
    "media_dias_recompra": 52.07692307692308
  },
  {
    "_id": "Ana Oliveira",
    "media_dias_recompra": 53
  }
... (382 linha(s) de resultado omitida(s) - total de 100 documento(s))

```

Figura 20 – Média de Dias para Recompra

Fonte: Elaborado pelo autor (2026)

O operador \$shift, utilizado dentro de \$setWindowFields, é o equivalente direto da função de janela LEAD() do T-SQL, permitindo acessar o valor de um campo em um documento subsequente dentro da mesma partição — neste caso, a data do próximo pedido do mesmo cliente.

```

1 db.pedidos.aggregate([

```

```

2    { $match: { pagamento: { $exists: true } } },
3    {
4      $facet: {
5        porEstado: [
6          { $group: { _id: "$cliente.estado_uf", faturamento: { $sum: "$p
pagamento.valor_pago" } } }
7        ],
8        totalGeral: [
9          { $group: { _id: null, total: { $sum: "$pagamento.valor_pago" }
} }
10       ]
11     }
12   },
13   { $unwind: "$porEstado" },
14   {
15     $project: {
16       estado_uf: "$porEstado._id",
17       percentual: {
18         $round: [
19           { $multiply: [
20             { $divide: ["$porEstado.faturamento", { $arrayElemAt: ["$t
otalGeral.total", 0] } ] },
21             100
22           ] }, 2
23         ]
24       }
25     }
26   },
27   { $sort: { percentual: -1 } }
28 ];

```

Algoritmo 20 – Market Share Estadual (SANTOS, 2026)

```

=====
Algoritmo 20 – Market Share Estadual (SANTOS, 2026)
=====
[
  {
    "estado_uf": "SP",
    "percentual": 27.65
  },
  {
    "estado_uf": "RJ",
    "percentual": 16.72
  },
  {
    "estado_uf": "PR",
    "percentual": 14.69
  },
  {
    "estado_uf": "SC",
    "percentual": 13.73
  },
  {
    "estado_uf": "MG",
    "percentual": 13.7
  },
  ... (6 linha(s) de resultado omitida(s) – total de 6 documento(s))
]

```

Figura 21 – Market Share Estadual

Fonte: Elaborado pelo autor (2026)

Assim como no Algoritmo 14, o estágio \$facet reproduz o efeito do CROSS JOIN entre a agregação por estado e o total geral de faturamento, permitindo comparar cada subtotal com o valor agregado global dentro de um único pipeline.

```

1 db.pedidos.aggregate([
2   { $match: { pagamento: { $exists: true } } },
3   {
4     $group: {
5       _id: "$cliente.estado_uf",
6       total_pago: { $sum: "$pagamento.valor_pago" },
7       qtd_pedidos: { $sum: 1 }
8     }
9   },
10  {
11    $project: {
12      _id: 1,
13      ticket_medio: {
14        $round: [{ $divide: ["$total_pago", "$qtd_pedidos"] }, 2]
15      }
16    }
17  },
18  { $sort: { ticket_medio: -1 } }
19 ]);

```

Algoritmo 21 – Ranking de Ticket Médio por Estado (SANTOS, 2026)

```

=====
Algoritmo 21 – Ranking de Ticket Médio por Estado (SANTOS, 2026)
=====
[
  {
    "_id": "SC",
    "ticket_medio": 715.27
  },
  {
    "_id": "RS",
    "ticket_medio": 684.33
  },
  {
    "_id": "SP",
    "ticket_medio": 672.57
  },
  {
    "_id": "PR",
    "ticket_medio": 659.67
  },
  {
    "_id": "RJ",
    "ticket_medio": 632.59
  }
... (6 linha(s) de resultado omitida(s) – total de 6 documento(s))

```

Figura 22 – Ranking de Ticket Médio por Estado

Fonte: Elaborado pelo autor (2026)

```

1 db.pedidos.aggregate([

```

```

2   { $match: { pagamento: { $exists: true } } },
3   {
4     $group: {
5       _id: {
6         ano: { $year: "$pagamento.data_pagamento" },
7         mes: { $month: "$pagamento.data_pagamento" }
8       },
9       faturamento: { $sum: "$pagamento.valor_pago" }
10    }
11  },
12  { $sort: { "_id.ano": 1, "_id.mes": 1 } },
13  {
14    $setWindowFields: {
15      sortBy: { "_id.ano": 1, "_id.mes": 1 },
16      output: {
17        faturamento_mes_anterior: { $shift: { output: "$faturamento",
by: -1 } }
18      }
19    }
20  },
21  {
22    $project: {
23      _id: 1,
24      faturamento: 1,
25      crescimento_bruto: { $subtract: ["$faturamento", "$faturamento_m
es_anterior"] }
26    }
27  }
28  });

```

Algoritmo 22 – Crescimento Financeiro Mês a Mês (SANTOS, 2026)

```

=====
Algoritmo 22 - Crescimento Financeiro Mês a Mês (SANTOS, 2026)
=====
[
  {
    "_id": {
      "ano": 2023,
      "mes": 10
    },
    "faturamento": 19058.2,
    "crescimento_bruto": null
  },
  {
    "_id": {
      "ano": 2023,
      "mes": 11
    },
    "faturamento": 9689.6,
    "crescimento_bruto": -9368.6
  },
  {
    "_id": {
      "ano": 2023,
... (254 linha(s) de resultado omitida(s) - total de 34 documento(s))

```

Figura 23 – Crescimento Financeiro Mês a Mês

Fonte: Elaborado pelo autor (2026)

Neste caso, o \$shift com deslocamento negativo (by: -1) reproduz a função LAG() do T-SQL, acessando o valor do documento anterior na partição ordenada.

```

1 db.pedidos.aggregate([
2   { $match: { status_pedido: { $in: ["Concluído", "Enviado"] } } },
3   { $unwind: "$itens" },
4   {
5     $group: {
6       _id: "$itens.nome_produto",
7       receita_gerada: {
8         $sum: { $multiply: ["$itens.quantidade", "$itens.preco_unit_ven
9         da"] }
10      }
11    },
12    { $sort: { receita_gerada: -1 } },
13    { $limit: 3 }
14  ]);

```

Algoritmo 23 – Os 3 Produtos com Maior Impacto Financeiro (SANTOS, 2026)

```

=====
Algoritmo 23 – Os 3 Produtos com Maior Impacto Financeiro (SANTOS, 2026)
=====
[
  {
    "_id": "Jaqueta Couro Sintético",
    "receita_gerada": 106369.59999999999
  },
  {
    "_id": "Calça Jeans Slim",
    "receita_gerada": 76817.3
  },
  {
    "_id": "Calça Sarja Cargo",
    "receita_gerada": 52573.700000000004
  }
]

```

Figura 24 – Os 3 Produtos com Maior Impacto Financeiro

Fonte: Elaborado pelo autor (2026)

```

1 db.pedidos.aggregate([
2   { $match: { pagamento: { $exists: true } } },
3   { $sort: { "pagamento.valor_pago": -1 } },
4   { $limit: 1 },
5   {
6     $project: {
7       _id: 0,
8       cliente_vip: "$cliente.nome",
9       maior_compra: "$pagamento.valor_pago"
10    }
11  }
12 ]);

```

Algoritmo 24 – A Maior Transação Única do Sistema (SANTOS, 2026)

```

=====
Algoritmo 24 – A Maior Transação Única do Sistema (SANTOS, 2026)
=====
[
  {
    "cliente_vip": "Henrique Lira",
    "maior_compra": 2639.0999999999995
  }
]

```

Figura 25 – A Maior Transação Única do Sistema

Fonte: Elaborado pelo autor (2026)

```

1  db.pedidos.aggregate([
2    { $match: { pagamento: { $exists: true } } },
3    {
4      $project: {
5        metodo: "$pagamento.metodo",
6        minutos_pagamento: {
7          $dateDiff: {
8            startDate: "$data_pedido",
9            endDate: "$pagamento.data_pagamento",
10           unit: "minute"
11         }
12       }
13     },
14   ],
15   {
16     $group: {
17       _id: "$metodo",
18       media_minutos_pagamento: { $avg: "$minutos_pagamento" }
19     }
20   }
21 ]);

```

Algoritmo 25 – Tempo Médio entre Pedido e Pagamento (SANTOS, 2026)

```

=====
Algoritmo 25 – Tempo Médio entre Pedido e Pagamento (SANTOS, 2026)
=====
[
  {
    "_id": "Boleto",
    "media_minutos_pagamento": 676.827027027027
  },
  {
    "_id": "PIX",
    "media_minutos_pagamento": 699.920704845815
  },
  {
    "_id": "Cartão de Débito",
    "media_minutos_pagamento": 705.2843601895735
  },
  {
    "_id": "Cartão de Crédito",
    "media_minutos_pagamento": 719.5990566037735
  }
]

```

Figura 26 – Tempo Médio entre Pedido e Pagamento

Fonte: Elaborado pelo autor (2026)

```

1 db.pedidos.aggregate([
2   { $unwind: "$itens" },
3   {
4     $lookup: {
5       from: "produtos",
6       localField: "itens.id_produto_legacy",
7       foreignField: "id_produto_legacy",
8       as: "produto_info"
9     }
10  },
11  { $unwind: "$produto_info" },
12  {
13    $group: {
14      _id: {
15        estado: "$cliente.estado_uf",
16        categoria: "$produto_info.categoria.nome_categoria"
17      },
18      qtd_comprada: { $sum: "$itens.quantidade" }
19    }
20  },
21  { $sort: { "_id.estado": 1, qtd_comprada: -1 } }
22 ]);

```

Algoritmo 26 – Matriz Demográfica: O que cada Estado Compra (SANTOS, 2026)

```

=====
Algoritmo 26 - Matriz Demográfica: O que cada Estado Compra (SANTOS, 2026)
=====
[
  {
    "_id": {
      "estado": "MG",
      "categoria": "Camisetas"
    },
    "qtd_comprada": 172
  },
  {
    "_id": {
      "estado": "MG",
      "categoria": "Calças e Bermudas"
    },
    "qtd_comprada": 163
  },
  {
    "_id": {
      "estado": "MG",
      "categoria": "Casacos e Jaquetas"
    },
    ... (192 linha(s) de resultado omitida(s) - total de 30 documento(s))
  }
]

```

Figura 27 – Matriz Demográfica: O que cada Estado Compra

Fonte: Elaborado pelo autor (2026)

```

1 db.pedidos.aggregate([
2   {
3     $group: {
4       _id: "$cliente.nome",
5       qtd_pedidos_total: { $sum: 1 }
6     }
7   },
8   { $sort: { qtd_pedidos_total: -1 } }
9 ]);

```

Algoritmo 27 – Frequência de Compras Repetidas por Cliente (SANTOS, 2026)

```
=====
Algoritmo 27 – Frequência de Compras Repetidas por Cliente (SANTOS, 2026)
=====
[
  {
    "_id": "Tatiana Campos",
    "qtd_pedidos_total": 20
  },
  {
    "_id": "Patricia Lima",
    "qtd_pedidos_total": 16
  },
  {
    "_id": "Letícia Martins",
    "qtd_pedidos_total": 16
  },
  {
    "_id": "Cesar Bueno",
    "qtd_pedidos_total": 15
  },
  {
    "_id": "Milena Furtado",
    "qtd_pedidos_total": 15
  }
]
... (382 linha(s) de resultado omitida(s) – total de 100 documento(s))
```

Figura 28 – Frequência de Compras Repetidas por Cliente

Fonte: Elaborado pelo autor (2026)

```
1 db.pedidos.aggregate([
2   { $match: { status_pedido: "Pendente" } },
3   { $unwind: "$itens" },
4   {
5     $group: {
6       _id: "$status_pedido",
7       valor_financeiro_pendente: {
8         $sum: { $multiply: ["$itens.quantidade", "$itens.preco_unit_ven
9       da"] }
10     }
11   }
12 ]);
```

Algoritmo 28 – Valor da Oportunidade Perdida: Pedidos Pendentes (SANTOS, 2026)

```
=====
Algoritmo 28 – Valor da Oportunidade Perdida: Pedidos Pendentes (SANTOS, 2026)
=====
[
  {
    "_id": "Pendente",
    "valor_financeiro_pendente": 69977.40000000001
  }
]
```

Figura 29 – Valor da Oportunidade Perdida: Pedidos Pendentes

Fonte: Elaborado pelo autor (2026)

```
1 db.produtos.aggregate([
```

```

2    {
3      $facet: {
4        totalCatalogo: [{ $count: "qtd" }],
5        disponiveis: [{ $match: { qtd_estoque: { $gt: 0 } } }, { $count:
"qtd" }],
6        rupturaEstoque: [{ $match: { qtd_estoque: 0 } }, { $count: "qtd"
}]
7      }
8    },
9    {
10     $project: {
11       total_catalogo: { $arrayElemAt: ["$totalCatalogo.qtd", 0] },
12       produtos_disponiveis: { $arrayElemAt: ["$disponiveis.qtd", 0] },
13       ruptura_de_estoque: { $arrayElemAt: ["$rupturaEstoque.qtd", 0] }
14     }
15   }
16 ];

```

Algoritmo 29 – Produtos Cadastrados vs. Disponíveis (SANTOS, 2026)

```

=====
Algoritmo 29 – Produtos Cadastrados vs. Disponíveis (SANTOS, 2026)
=====
[
  {
    "total_catalogo": 30,
    "produtos_disponiveis": 27,
    "ruptura_de_estoque": 3
  }
]

```

Figura 30 – Produtos Cadastrados vs. Disponíveis

Fonte: Elaborado pelo autor (2026)

```

1  db.pedidos.aggregate([
2    { $sort: { data_pedido: -1 } },
3    { $limit: 20 },
4    { $unwind: "$itens" },
5    {
6      $project: {
7        _id: 0,
8        id_pedido_legado: 1,
9        nome_cliente: "$cliente.nome",
10       nome_produto: "$itens.nome_produto",
11       quantidade: "$itens.quantidade",
12       metodo_pag: { $ifNull: ["$pagamento.metodo", null] },
13       status_pedido: 1
14     }
15   }
16 ]);

```

Algoritmo 30 – Visão Desnormalizada de Auditoria (SANTOS, 2026)

```

=====
Algoritmo 30 – Visão Desnormalizada de Auditoria (SANTOS, 2026)
=====
[
  {
    "id_pedido_legado": 815,
    "status_pedido": "Concluído",
    "nome_cliente": "Fernando Braga",
    "nome_produto": "Kit 3 Cuecas Boxer",
    "quantidade": 1,
    "metodo_pag": "PIX"
  },
  {
    "id_pedido_legado": 815,
    "status_pedido": "Concluído",
    "nome_cliente": "Fernando Braga",
    "nome_produto": "Jaqueta Couro Sintético",
    "quantidade": 3,
    "metodo_pag": "PIX"
  },
  {
    "id_pedido_legado": 815,
    "status_pedido": "Concluído",
    ... (366 linha(s) de resultado omitida(s) – total de 48 documento(s))
  }
]

```

Figura 31 – Visão Desnormalizada de Auditoria

Fonte: Elaborado pelo autor (2026)

Vale destacar que esta última consulta, embora nomeada "visão desnormalizada" no projeto relacional original — onde exigia quatro operações de junção (INNER JOIN com CLIENTE, ITEM_PEDIDO, PRODUTO e LEFT JOIN com PAGAMENTO) —, torna-se, no modelo de documentos, uma simples projeção sobre um único documento já nativamente desnormalizado, sintetizando de forma exemplar a proposta central deste trabalho: o modelo de dados não relacional aproxima a estrutura de armazenamento da estrutura de consumo da aplicação.

5 CONCLUSÃO

O desenvolvimento deste projeto permitiu consolidar, na prática, a compreensão das diferenças estruturais e filosóficas entre o modelo relacional e o modelo de dados orientado a documentos. Retomando os objetivos específicos estabelecidos na introdução, verifica-se que todos foram integralmente alcançados: (i) o estudo comparativo dos modelos NoSQL evidenciou a adequação do modelo orientado a documentos ao domínio de e-commerce, especialmente pela coesão natural entre pedido, itens e pagamento; (ii) as seis entidades relacionais normalizadas foram transpostas com sucesso para três coleções — clientes, produtos e pedidos —, aplicando de forma criteriosa as estratégias de incorporação

e referenciamento conforme o padrão de acesso de cada relacionamento; (iii) a carga de dados foi reproduzida com a mesma volumetria do projeto relacional anterior, garantindo comparabilidade entre os dois estudos de caso; e (iv) as 30 consultas analíticas foram integralmente adaptadas para o Aggregation Framework do MongoDB, demonstrando que operadores como \$lookup, \$facet, \$unionWith e \$setWindowFields são capazes de reproduzir, com sintaxe própria, praticamente todo o repertório de junções, subconsultas correlacionadas e funções de janela originalmente disponíveis em T-SQL.

A comparação entre os dois paradigmas evidencia vantagens e desvantagens claras. Como vantagens do modelo NoSQL orientado a documentos observadas neste projeto, destacam-se: a significativa simplificação de consultas que operam sobre a unidade "pedido completo", eliminando junções antes necessárias entre três tabelas distintas (Algoritmos 6 e 30); a flexibilidade de esquema, que permitiria, por exemplo, a adição de novos atributos de produto (como composição têxtil ou tabela de medidas) sem necessidade de alteração de esquema (ALTER TABLE); e o potencial de escalabilidade horizontal por meio de sharding, não contemplado neste estudo de caso, mas nativamente suportado pela arquitetura do MongoDB.

Como desvantagens, destacam-se: a maior complexidade de consultas que exigem agregações cruzando múltiplas coleções, como a matriz demográfica de produtos por estado (Algoritmo 26), que ainda depende de \$lookup e \$unwind para desaninhar arrays; a ausência de garantias transacionais ACID nativas entre documentos de coleções distintas na configuração padrão utilizada (embora o MongoDB ofereça suporte a transações multidocumento desde a versão 4.0, seu uso introduz sobrecarga de desempenho e foi deliberadamente evitado neste projeto, coerente com a filosofia de uso do banco); e o risco de inconsistência de dados replicados, como o nome do produto e o nome do cliente incorporados no documento de pedido, que não se atualizam automaticamente caso o dado de origem seja alterado, exigindo rotinas de sincronização (por exemplo, via triggers de aplicação ou change streams) caso a atualização histórica seja um requisito de negócio.

Comparativamente ao Microsoft SQL Server utilizado no projeto relacional anterior, conclui-se que a escolha entre os dois paradigmas não deve ser tratada como uma substituição definitiva, mas como uma decisão de engenharia dependente do padrão de acesso predominante da aplicação: para operações

transacionais que exigem forte consistência entre múltiplas entidades (como reconciliação financeira e controle de estoque em tempo real), o modelo relacional permanece mais robusto; para operações de leitura intensiva sobre agregados de negócio coesos (como a exibição do histórico de pedidos de um cliente), o modelo de documentos oferece ganhos claros de simplicidade e performance.

Sugestões para Possíveis Melhorias:

Para ciclos futuros de evolução deste projeto, propõem-se as seguintes melhorias:

- Implementação de Change Streams: utilização do recurso de Change Streams do MongoDB para propagar automaticamente atualizações de nome de cliente ou preço de produto para os documentos de pedido já incorporados, mitigando o risco de inconsistência de dados desnormalizados;
- Adoção de Transações Multidocumento: avaliar o uso de sessões transacionais (startSession) nas rotinas de inserção de pedido e pagamento em cenários que exijam consistência estrita, ponderando o impacto no desempenho;
- Criação de Índices Compostos: implementar índices compostos sobre os campos status_pedido, data_pedido e cliente.estado_uf, otimizando as consultas de agregação mais recorrentes identificadas neste trabalho;
- Estratégia Híbrida (Polyglot Persistence): considerar, para trabalhos futuros, a coexistência do MongoDB com um banco relacional para o subdomínio financeiro, explorando a abordagem de persistência poliglota defendida por Sadalage e Fowler (2013);
- Testes de Carga e Sharding: conduzir testes de desempenho comparativos entre os dois SGBDs sob volumetria crescente, avaliando o comportamento do sharding horizontal do MongoDB frente ao particionamento de tabelas do SQL Server.

REFERÊNCIAS

BREWER, Eric A. Towards robust distributed systems. Proceedings of the 19th Annual ACM Symposium on Principles of Distributed Computing, Portland, 2000.

CHODOROW, Kristina. MongoDB: The Definitive Guide. 2. ed. Sebastopol: O'Reilly Media, 2013.

DATE, C. J. Introdução a Sistemas de Bancos de Dados. 8. ed. Rio de Janeiro: Elsevier, 2004.

ELMASRI, Ramez; NAVATHE, Shamkant B. Sistemas de Banco de Dados. 6. ed. São Paulo: Pearson Addison-Wesley, 2011.

HEUSER, Carlos Alberto. Projeto de Banco de Dados. 6. ed. Porto Alegre: Bookman, 2009.

MONGODB, Inc. MongoDB Manual: Aggregation Pipeline. Disponível em: <https://www.mongodb.com/docs/manual/aggregation/>. Acesso em: 30 jun. 2026.

MONGODB, Inc. MongoDB Manual: Data Modeling. Disponível em: <https://www.mongodb.com/docs/manual/core/data-modeling-introduction/>. Acesso em: 30 jun. 2026.

SADALAGE, Pramod J.; FOWLER, Martin. NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Boston: Addison-Wesley, 2013.

SANTOS, Vinícius de Moraes Almeida. Sistema de Gerenciamento de Banco de Dados para um E-commerce de Vestuário. Campos do Jordão: IFSP, 2026. Projeto Final de Banco de Dados Relacional.